In Class Activity 09 Critical Thinking

Gowtham Revanur

Panther ID: 002574540

Pathway: Graduate

Assignment: Local Storage Part II

Prompt 1 Failure reconstruction

If a classmate sees zero cards after the version 2 upgrade while the Part I table still has rows, one cause can be a migration problem: the opened database filename may not be MyDatabase.db, or the installed database version may not increase from 1 to 2, so the expected tables were never added. A second cause can be a query or UI problem: the cards screen may query the wrong folder_id or fail to refresh after a successful write. I would distinguish these causes by logging the opened path, user_version, folder ID, query arguments, and returned row count. The safest next step is a read-only inspection of the existing database and arguments. I would not clear storage, uninstall, reseed, or recreate the Part I table because those actions destroy the evidence needed to diagnose the problem.

In my implementation, DatabaseHelper keeps MyDatabase.db and adds version 2 through onUpgrade. The card query uses where: folder_id = ? with the selected integer folder ID. Runtime T1 evidence still needs to be recorded on the Android installation before I claim the migration worked on that device.

Prompt 2 Delete policy trade off

ON DELETE CASCADE is appropriate for a disposable card catalogue because a card cannot remain meaningful without its folder. It would be a poor default for a business system such as invoices and customer records: deleting a customer should not erase financial history. A safer policy there would restrict deletion while dependent invoices exist, or mark the customer inactive. The UI safeguard would show the number and names of dependent records and require a deliberate archival action. The corresponding test would attempt deletion with dependent invoices and verify that no invoice disappears.

For this lab, the confirmation names the folder and reports the number of child cards. Foreign-key enforcement is enabled in onConfigure, rather than relying on the cascade declaration alone. T5 must still be executed with fictional disposable data and recorded with the actual observed IDs and counts.

Prompt 3 Image forensics

My decision tree begins by reading image_ref. If it is null or blank, the app shows the suit-symbol placeholder. If it names a bundled asset, I check whether the path is registered in pubspec.yaml; a missing asset should show the placeholder while the card title, suit, notes, and database row remain visible. If it is an app-private path, I check whether the file still exists; a deleted file should not cause deletion of the card row. If it is a URL, an unavailable network response should leave the card usable with the placeholder. If the reference has no supported form, I treat it as malformed and preserve the record while reporting the fallback state.

This submission intentionally stores only a nullable image reference and uses the suit symbol as its stable fallback. No gallery picker or remote-image dependency is required.

Prompt 4 Migration design defence

For version 3, I would add favourite as an INTEGER NOT NULL DEFAULT 0 and review_date as nullable TEXT. onUpgrade would run only when oldVersion is less than 3 and would perform both ALTER TABLE statements in a transaction. Existing cards receive favourite = 0 through the default and review_date = NULL, so existing meaning is preserved. If either statement fails, the transaction rolls back and the app must show a database initialization error instead of pretending the schema is current.

Two tests would verify that an existing version-2 card remains present with favourite false and a null review date, and that a newly edited favourite/review date survives a restart. I reject the destructive shortcut of dropping and recreating cards because it would erase card IDs, folder relationships, notes, and evidence from existing users.

AI Alchemist Lab G1

Claim evaluated: A foreign key with ON DELETE CASCADE always protects child rows correctly. Prediction: this is false unless foreign-key enforcement is enabled for the SQLite connection. Investigation: the database initialization includes PRAGMA foreign_keys = ON in onConfigure and declares cards.folder_id as a foreign key with ON DELETE CASCADE. This supports the schema design, but it does not replace the required T5 runtime observation. Counterexample: a connection that never enables foreign keys can accept the declaration while failing to cascade. Safer design: keep onConfigure, test deletion only with fictional data, and record the affected child count before and after deletion. A falsifiable follow-up test is to create a folder and cards, delete the folder, and query cards by the prior folder ID; any remaining row disproves the cascade result.

AI disclosure: Codex was used on October 8, 2026 to critique the migration design and organize this response. I reviewed the generated source and will verify all runtime claims on my own Android test run.

AI Alchemist Lab G2

Claim evaluated: If a form validates a card title, the database cannot receive an invalid card. Prediction: this is only partly true because another code path could write directly to SQLite. Investigation: the UI rejects a blank trimmed title before calling insertCard or updateCard, while the database also stores title as NOT NULL. This evidence supports protection against empty UI submissions, but it does not prove every non-UI writer applies the same business rule. Counterexample: a repository caller could pass whitespace unless a shared validator or database CHECK constraint exists. Safer design: retain field-level validation and add a database constraint or shared domain validator. A falsifiable follow-up test is to call the repository with whitespace through a controlled test and verify the write is rejected without changing the count.

AI disclosure: Codex was used on October 8, 2026 to critique this hypothesis. No private code, credentials, or personal test data were shared.

Evidence status

The source and release APK were statically verified with flutter analyze reporting No issues found. Required Android T1 through T6 screenshots and observed IDs/counts are not invented in this document. They must be added after an actual emulator or device run before final upload.